memory_layers_agent — Dual-Layer Memory (Session Short-Term vs Memory Long-Term) Agent

`memory_layers_agent` is an intelligent assistant designed to practice and verify Google Agent Development Kit (ADK)'s **Dual-Layer Memory** architecture. It combines the **Session Service (short-term memory)**, which manages active conversation threads and transient states, with the **Memory Service (long-term memory)**, which persists and retrieves facts and user knowledge across sessions, enabling personalized, continuous multi-session conversations.

> **Related Lecture Note**: Part 2 · 02. Google ADK Architecture (Slide 29: Dual-Layer Memory)

---

1. Dual-Layer Memory Comparison Matrix

Dimension	Session Service (Short-term Memory)	Memory Service (Long-term Memory)
Storage Scope	Active conversation thread and transient session state	Persistent user facts, preferences, and knowledge across sessions
Data Stored	Message history, tool execution results, State variables	Structured/extracted user profile, past events, and preference records
Lifecycle	Destroyed when conversation thread ends (preserved in DB if using Database session)	Persisted permanently across sessions
Implementations	`InMemorySessionService`, `DatabaseSessionService`, `VertexAiSessionService`	`InMemoryMemoryService`, `VertexAiMemoryBankService`
Agent Access Method	Event history auto-injection, `{key}` instruction template, `tool_context.state`	`preload_memory` (automatic per-turn pre-search), `load_memory` (active retrieval tool)

---

2. Session State Variable Prefix Rules

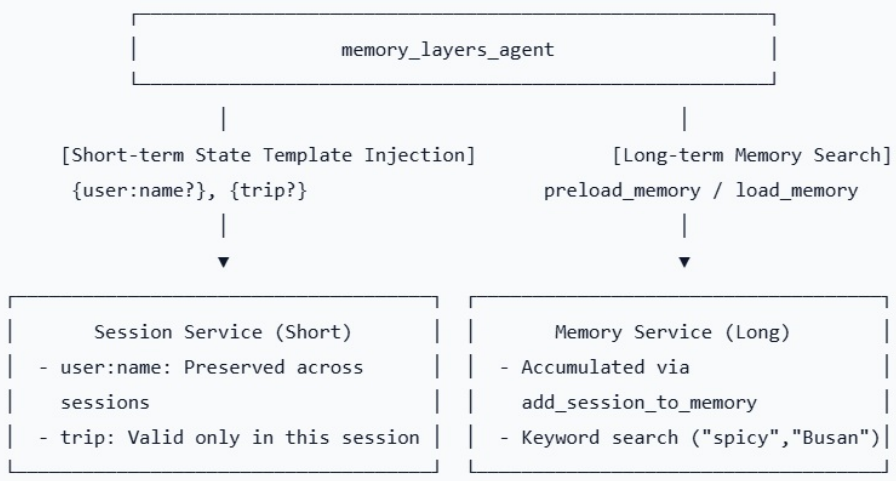
In Google ADK, the storage scope and sharing boundary of Session State variables are strictly determined by their key prefixes:

- `user:<key>` (e.g., `user:name`):
  - **User Scope**: Automatically shared and preserved across **all conversation sessions** created by the same user (`user_id`).
  - Best suited for persistent profile data such as user names, job titles, and default settings.
- **No prefix** (e.g., `trip`):
  - **Session Scope**: Valid only within that specific conversation session (`session_id`). Completely reset when a new session begins.
  - Best suited for transient context such as one-off trip plans or in-progress order IDs.
- `app:<key>` (e.g., `app:notice`):
  - **Application Global Scope**: Shared globally across all users and all sessions within the application (`app_name`).
- `temp:<key>` (e.g., `temp:raw`):
  - **Invocation Turn Scope**: Kept in memory temporarily only during a single invocation turn; never persisted to session storage (such as a database).

---

3. Agent Architecture and Tool Configuration

`memory_layers_agent` leverages both memory layers through instruction template variable substitution and dedicated tools:



① Instruction Template

```
instruction="""
You are a personal assistant.
[Short-term: This Session State]
  User Name: {user:name?}
  Plan for this session: {trip?}
Rules:
1) If the user states their name, call save_profile; if they mention a trip/business plan, call
save_trip.
2) For past information not in this session State (food preferences, plans from previous conversations,
etc.),
   check the automatically provided past memories first; if insufficient, search with load_memory.
   Use single core keywords without postpositions for load_memory (e.g., "spicy", "Busan", "trip").
3) If not found even in memory, say you do not know. Do not hallucinate.
4) Keep responses within 2-3 sentences.
"""
```

- `{user:name?}` : If `user:name` exists in the session State, its value is injected; if absent, the question mark (`?`) safely substitutes it with an empty string.
- `{trip?}` : Automatically injects plan information recorded during the current session into the prompt.

② Tool Configuration

1. `save_profile(name, tool_context)` :
   - `tool_context.state["user:name"] = name`
   - Saves the user's name into a persistent `user:` scoped State variable, instantly accessible across other sessions.
2. `save_trip(trip, tool_context)` :
   - `tool_context.state["trip"] = trip`
   - Saves trip/travel plans into the current session-scoped State variable.
3. `preload_memory` :
   - Official Google ADK built-in tool. Automatically searches the Memory Service using keywords from the user's query each turn and injects relevant context into the prompt.
4. `load_memory` :
   - Official Google ADK built-in tool. Actively queries long-term memory on-demand when the model decides additional past context is required.

---

4. File Structure

```
memory_layers_agent/
├── __init__.py          # Exposes agent module and root_agent object
├── agent.py             # Defines save_profile, save_trip tools, instruction, and root_agent
├── README-KR.md         # Korean detailed documentation
└── README-EN.md         # English translated documentation (this document)
```

- `agent.py` :
  - `save_profile(name: str, tool_context: ToolContext) -> dict` : Saves user name (`user:name`)
  - `save_trip(trip: str, tool_context: ToolContext) -> dict` : Saves session itinerary (`trip`)
  - `preload_memory` , `load_memory` : Registers ADK official memory tools
  - `root_agent` : Gemini-powered agent instance
- `__init__.py` :
  - Exposes `from .agent import root_agent` as the entrypoint for the ADK CLI

5. Prerequisites and Configuration

1) Package Installation

```
pip install google-adk python-dotenv
# or using uv
uv pip install google-adk python-dotenv
```

2) Environment Variables (`.env`) Configuration

Create a `.env` file in the parent directory (`part02_architecture` or project root):

```
# Gemini Developer API Key
GEMINI_API_KEY=your_gemini_api_key_here

# Or when using Google Cloud Vertex AI
# GOOGLE_GENAI_USE_VERTEXAI=TRUE
# GOOGLE_CLOUD_PROJECT=your-gcp-project-id
# GOOGLE_CLOUD_LOCATION=us-central1

# Agent model configuration
ADK_MODEL=gemini-flash-latest
```

6. Execution and Testing Methods

Method 1: Terminal Interactive CLI Mode (`adk run`)

Directly interact with the agent in your terminal to inspect session State updates:

```
# Run from within the adk-based_Agent folder
adk run memory_layers_agent
```

Or run from the parent `part02_architecture` folder:

```
cd ..
adk run adk-based_Agent/memory_layers_agent
```

Interact with the agent as follows:

```
User: Hello, my name is John and I have a business trip to Jeju next Monday.
Agent: Nice to meet you, John. I will remember your business trip to Jeju next Monday.
```

Method 2: Web Development UI Environment (`adk web`)

Use Google ADK's web-based GUI to converse and inspect session State variables and event streams in real time:

```
# Run from within the adk-based_Agent folder
adk web --port 8008
```

Navigate to `http://localhost:8008` in your browser and select `memory_layers_agent` from the top-left dropdown. You can visually observe tool execution parameters and how `state_delta` accumulates.

Method 3: Memory Layers Dedicated Verification Script (`run_memory_layers.py`)

Run the verification script in the parent `Direct_ADK` folder to step-by-step verify storage scopes, lifecycles, and retrieval mechanisms for both Session and Memory services:

```
# Run from part02_architecture directory

# 1) Offline mode: Verify storage structure and lifecycles via Service API without LLM calls
uv run Direct_ADK/run_memory_layers.py offline

# 2) LLM integration mode: Save in Session 1 ➡️ Retrieve long-term memory in Session 2 via Gemini
uv run Direct_ADK/run_memory_layers.py llm

# 3) Run both modes (default)
uv run Direct_ADK/run_memory_layers.py all
```

Method 4: Multi-Session Memory Verification via Python Code

Example Python script demonstrating long-term memory transition across two consecutive sessions:

```
import asyncio
from google.adk.sessions import InMemorySessionService
from google.adk.memory import InMemoryMemoryService
from common.runner_util import Session
from memory_layers_agent.agent import root_agent

async def main():
    session_service = InMemorySessionService()
    memory_service = InMemoryMemoryService()
    app, user = "memory_demo", "user_john"

    # [Session 1] Save personal profile and preferences
    print("=== Starting Session 1 ===")
    s1 = Session(root_agent, app_name=app, user_id=user,
                 session_service=session_service, memory_service=memory_service)
    await s1.send("Hello! I am John. I cannot eat spicy food at all. I have a business trip to Busan
next Tuesday.")

    # Transfer session history to long-term memory upon session completion
    current_s1 = await s1.get_session()
    await memory_service.add_session_to_memory(current_s1)
    print("Session 1 State:", current_s1.state)

    # [Session 2] Start a new session (New session ID)
    print("\n=== Starting Session 2 (New Conversation Thread) ===")
    s2 = Session(root_agent, app_name=app, user_id=user,
                 session_service=session_service, memory_service=memory_service)
    # Include 'Busan' keyword to trigger preload_memory keyword match
    res = await s2.send("What was my name? And please recommend a dinner menu for my business trip to
Busan.")
    print("Agent Response:", res)
    print("Session 2 State:", (await s2.get_session()).state)

if __name__ == "__main__":
    asyncio.run(main())
```

---

7. Conversation Scenarios and Expected Output

[Session 1] Providing Profile, Short-term Itinerary, and Preferences

- **User**: "Hello, I am John. I cannot eat spicy food, and I have a business trip to Busan next Tuesday."
- **Agent Internal Workflow**:
  1. Calls `save_profile(name="John")` ➡️ Saves `state["user:name"] = "John"` (User-wide scope)
  2. `state["trip"] = "Busan business trip next Tuesday"`
  3. Upon session conclusion, the full conversation is transferred to long-term memory via `memory_service.add_session_to_memory(session)`

[Session 2] New Conversation Thread (New Session ID)

- **State Comparison**:
  - The `trip` variable was destroyed with the end of Session 1 and does not exist in Session 2.
  - The `user:name` variable carries the user scope (`user:`) and is automatically retained in the new session State.
- **User**: "What was my name? And please recommend a dinner menu for my business trip to Busan."
- **Agent Internal Workflow**:
  1. Instantly recognizes the user's name "John" from session State via the instruction template `{user:name?}` .
  2. Powered by the keyword 'Busan', `preload_memory` and `load_memory` search and retrieve the past preference ("cannot eat spicy food") from long-term memory.
- **Agent Final Output**:

> "John, since you should avoid spicy food, I recommend mild and hearty Busan pork soup (Dwaeji Gukbap) or fresh grilled fish for your dinner."

---

8. Troubleshooting and FAQ

Symptom	Root Cause	Resolution
User name forgotten in a new session	Saved without `user:` prefix, or session service instance was recreated	Ensure the `save_profile` tool assigns to `tool_context.state["user:name"]`.
Past preferences (e.g., spicy food intolerance) not found in a new session	`memory_service.add_session_to_memory(s1)` was not called at the end of Session 1	Verify that the session conclusion handler calls `add_session_to_memory`.
`InMemoryMemoryService` search returns empty	The `InMemory` implementation uses exact substring keyword matching	Ensure queries contain the exact keyword (e.g., "Busan", "spicy") present in memory. In production, use `VertexAiMemoryBankService` for semantic vector search.